Supervised Learning and k-Nearest Neighbors

Graduate-Level Lecture Notes / Handout

1 Introduction to Supervised Learning

Definition 1 (Supervised Learning)**.** *Supervised learning is a branch of machine learning where an algorithm learns a mapping from input variables (features)* $\mathbf{x} \in \mathcal{X}^d$ *to output variables (labels)* $y \in \mathcal{Y}$*, based on a set of labeled examples. The training set provides supervision in the form of true labels.*

- **Regression:** The label space $\mathcal{Y} \subseteq \mathbb{R}$ is continuous.

  Example 1. *Predicting house prices based on features such as area, number of rooms, and location.*

- **Classification:** The label space is discrete, $\mathcal{Y} = \{1, 2, \ldots, C\}$.

  Example 2. *Predicting whether a given email is* spam *(1) or* not spam *(0).*

2 Supervised Learning Setup

We observe training data

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\} \subseteq \mathcal{X}^d \times \mathcal{Y},$$

where $\mathbf{x}_i \in \mathcal{X}^d$ is a feature vector and $y_i \in \mathcal{Y}$ is a label.

Goal: Find a hypothesis $h : \mathcal{X}^d \to \mathcal{Y}$ such that predictions $h(\mathbf{x})$ are close to true labels y for unseen data.

3 Risk Minimization Principle

3.1 True Risk

$$R(h) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\big[L(h(\mathbf{x}), y)\big],$$

where $\mathcal{D}$ is the (unknown) joint distribution of data and L is a loss function.

3.2 Empirical Risk

$$\hat{R}_n(h) = \frac{1}{n}\sum_{i=1}^{n} L(h(\mathbf{x}_i), y_i).$$

3.3 Empirical Risk Minimization (ERM)

$$h_{\mathrm{ERM}} = \arg\min_{h\in\mathcal{H}} \hat{R}_n(h).$$

3.4 Regularized Risk Minimization

$$h^* = \arg\min_{h \in \mathcal{H}} \Big[\hat{R}_n(h) + \lambda\, \Omega(h) \Big],$$

where $\lambda > 0$ controls the trade-off between fitting training data and model complexity.

Example 3. *In ridge regression,*

$$\Omega(h) = \|w\|_2^2,$$

penalizing large weights in the linear model.

4 Bias and Variance

Definition 2 (Bias)**.** *Bias measures how far the expected prediction of the model is from the true function:*

$$Bias(x) = \mathbb{E}[\hat{h}(x)] - f(x).$$

Example 4. *A linear model applied to quadratic data has high bias because it cannot capture curvature.*

Definition 3 (Variance)**.** *Variance measures how sensitive the model is to variations in training data:*

$$Var(x) = \mathbb{E}\big[(\hat{h}(x) - \mathbb{E}[\hat{h}(x)])^2\big].$$

Example 5. *A decision tree with no pruning has high variance because small changes in training data drastically change predictions.*

4.1 Bias-Variance Decomposition

$$\mathbb{E}[(y - \hat{h}(x))^2] = (\text{Bias}(x))^2 + \text{Var}(x) + \sigma^2,$$

where σ^2 is irreducible noise.

4.2 Bias-Variance Trade-off

Why it happens: Simple models (e.g., linear regression) underfit, leading to **high bias** but **low variance**. Complex models (e.g., deep neural networks, unpruned trees) fit closely, giving **low bias** but **high variance**.

How it happens: As model complexity increases, bias decreases but variance increases, since the model becomes sensitive to random noise.

Impact: There exists an optimal point in complexity where the total error (bias^2 + variance + noise) is minimized. Regularization, ensembling, and cross-validation help manage this trade-off.

5 k-Nearest Neighbors (kNN)

Definition 4 (kNN Classifier)**.** *Given a query point* $\mathbf{x}$*:*

1. *Compute distances* $d(\mathbf{x}, \mathbf{x}_i)$ *to all training points.*
2. *Select* k *nearest neighbors* $\mathcal{N}_k(\mathbf{x})$*.*
3. *Predict majority class:*

$$h(\mathbf{x}) = \arg\max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}_{\{y_i = c\}}.$$

Properties:

- Non-parametric, instance-based learning.
- Hyperparameters: k, distance metric (Euclidean, Manhattan, Cosine, etc.).
- Small k: high variance (sensitive to noise).
- Large k: high bias (oversmoothing).

Remark 1. *In high dimensions, distances between points become less meaningful. This is the* curse of dimensionality*.*

6 Classifier Performance Evaluation

6.1 Confusion Matrix

A **confusion matrix** describes classification performance:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

6.2 Evaluation Metrics

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Fraction of predictions that are correct. *Use case:* Overall measure of correctness, but misleading with class imbalance.

Precision (Positive Predictive Value):

$$\text{Precision} = \frac{TP}{TP + FP}$$

How many predicted positives are truly positive. *Use case:* Important when false positives are costly (spam filters, fraud detection).

Recall (Sensitivity, True Positive Rate):

$$\text{Recall} = \frac{TP}{TP + FN}$$

How many actual positives are correctly identified. *Use case:* Crucial in medical screening where missing a case is unacceptable.

Specificity (True Negative Rate):

$$\text{Specificity} = \frac{TN}{TN + FP}$$

How many actual negatives are correctly identified. *Use case:* Important where false alarms are costly (fraud detection, quality control).

F1 Score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Balances precision and recall. *Use case:* Effective when both false positives and false negatives matter (search engines, IR).

6.3 Visualization

$$\begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix}$$

7 Linear Algebra Essentials for Machine Learning

This section reviews key linear algebra concepts relevant to supervised learning.

7.1 Systems of Linear Equations

A system of linear equations can be written in matrix form:

$$A\mathbf{x} = \mathbf{b},$$

where $A \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, and $\mathbf{b} \in \mathbb{R}^m$. - If $m = n$ and A is invertible, there is a unique solution $\mathbf{x} = A^{-1}\mathbf{b}$. - If $m \neq n$, or A is not invertible, we consider approximate solutions (least squares, pseudoinverse).

7.2 Span and Basis

Definition 5 (Span)**.** *The* span *of a set of vectors* $\{v_1, v_2, \ldots, v_k\}$ *is the set of all linear combinations:*

$$\mathit{span}\{v_1, \ldots, v_k\} = \left\{ \sum_{i=1}^{k} \alpha_i v_i : \alpha_i \in \mathbb{R} \right\}.$$

Definition 6 (Basis)**.** *A set of vectors* $\{v_1, \ldots, v_k\}$ *forms a* basis *of a vector space* V *if: 1. The vectors are linearly independent. 2. Their span covers* V*.*

7.3 Linear Independence

Definition 7 (Linear Independence)**.** *Vectors* $\{v_1, \ldots, v_k\}$ *are linearly independent if the only solution to*

$$\alpha_1 v_1 + \cdots + \alpha_k v_k = 0$$

is $\alpha_1 = \cdots = \alpha_k = 0$.

If they are linearly dependent, one vector is redundant and can be expressed as a combination of others.

7.4 Matrix Inverse

A square matrix $A \in \mathbb{R}^{n \times n}$ is invertible if there exists A^{-1} such that

$$AA^{-1} = A^{-1}A = I.$$

- An inverse exists if and only if A has full rank (all rows and columns are linearly independent).
- If rows or columns are linearly dependent, the matrix is singular and no inverse exists.

7.5 Pseudoinverse

For non-square or singular matrices, we use the *Moore–Penrose pseudoinverse* A^+, defined so that:

$$A^+ = \arg\min_X \|AX - I\|_F.$$

- If A is invertible, then $A^+ = A^{-1}$.
- If A has linearly dependent columns, A^+ gives the minimum-norm least-squares solution.

7.6 Effect of Independent Rows and Columns

- **Independent rows:** Full row rank ($\text{rank}(A) = m$) means equations are consistent (no redundancy).
- **Independent columns:** Full column rank ($\text{rank}(A) = n$) means variables are non-redundant, allowing unique solutions.
- A guaranteed inverse requires both row and column independence (i.e., full rank and square A).

7.7 Guaranteed Inverse

Theorem 1. *A matrix* $A \in \mathbb{R}^{n \times n}$ *is guaranteed to have an inverse if and only if* $\det(A) \neq 0$ *(equivalently, rank*$(A) = n$*).*

7.8 Gram Matrix and Pseudoinverse

Given $A \in \mathbb{R}^{m \times n}$: - The *Gram matrix* is $G = A^\top A \in \mathbb{R}^{n \times n}$, symmetric and positive semidefinite. - If A has full column rank, the pseudoinverse is:

$$A^+ = (A^\top A)^{-1} A^\top.$$

- The Gram matrix plays a central role in least squares regression, kernel methods, and support vector machines.

7.9 Types of Pseudoinverses

Left Inverse. A matrix $A \in \mathbb{R}^{m \times n}$ has a *left inverse* $A_L^{-1} \in \mathbb{R}^{n \times m}$ if

$$A_L^{-1} A = I_n.$$

Condition: Exists if A has full column rank ($\text{rank}(A) = n$ with $m \geq n$). **Interpretation:** Solves $A\mathbf{x} = \mathbf{b}$ uniquely when columns are linearly independent.

Right Inverse. A matrix $A \in \mathbb{R}^{m \times n}$ has a *right inverse* $A_R^{-1} \in \mathbb{R}^{n \times m}$ if

$$A A_R^{-1} = I_m.$$

Condition: Exists if A has full row rank ($\text{rank}(A) = m$ with $n \geq m$). **Interpretation:** Ensures consistency of solutions when rows are linearly independent.

Moore–Penrose Pseudoinverse. The *Moore–Penrose pseudoinverse* $A^+ \in \mathbb{R}^{n \times m}$ is the unique matrix satisfying the four Penrose conditions:

1. $AA^+A = A$,
2. $A^+AA^+ = A^+$,
3. $(AA^+)^\top = AA^+$,
4. $(A^+A)^\top = A^+A$.

Special cases:

- If A has full column rank ($m \geq n$), then $A^+ = (A^\top A)^{-1} A^\top$.
- If A has full row rank ($n \geq m$), then $A^+ = A^\top (AA^\top)^{-1}$.
- If A is square and invertible, $A^+ = A^{-1}$.

Interpretation: Provides the minimum-norm least-squares solution to $A\mathbf{x} \approx \mathbf{b}$.